# Information System Design — Assignment #1

School of Computer Science & IT, DAVV, Indore

Theory / analysis answers  Design / diagram answers

## Q1 Identify the different components in: Library Information System, Microsoft Office, Cellular Phone.

| Component type | Library Information System | Microsoft Office | Cellular Phone |
|---|---|---|---|
| **Hardware** | Servers, PCs, barcode scanners, printers | User's PC / laptop / tablet | Processor (SoC), memory, display, battery, camera, sensors |
| **Software** | Library software, **DBMS**, OPAC | **Word, Excel, PowerPoint, Outlook, Access** | **OS** (Android/iOS), firmware, apps |
| **Data** | Book catalogue, member & transaction records | Documents, sheets, slides, templates | Contacts, messages, media, settings |
| **People** | Librarian, members, administrator | End users, IT admin | User, service provider |
| **Procedures** | Issue/return rules, fine policy | Licensing, update & file-format rules | Calling, SMS, data-usage plans |
| **Network / Comm.** | LAN / Internet access | Cloud (OneDrive), email servers | **Antenna, SIM, baseband**, cell towers |
| **User interface** | Search & circulation screens | Ribbon, menus, shared dialogs | Touch screen, keypad, voice |

- **Common idea:** every system = **inputs + processing + outputs + feedback**, built from interacting components.

## Q2 How accurate is the COCOMO model in estimating software development effort?

- **COCOMO** (Boehm, 1981) estimates **effort = a × (KLOC)$^{b}$** person-months; modes: **Organic (2.4, 1.05)**, **Semi-detached (3.0, 1.12)**, **Embedded (3.6, 1.20)**.
- **Accuracy:** only **moderate**. Boehm reported Intermediate COCOMO within about **20% of actuals ~68% of the time** on its own data; Basic COCOMO is much rougher.
- Un-calibrated use on other organisations often gives errors of **50–100% or more**; **calibration with local data** improves it a lot.

| Why it is reasonably good | Why it can be inaccurate |
|---|---|
| Based on real project data; simple and transparent | **Needs KLOC early** – size estimate itself is uncertain |
| Cost drivers (15 in Intermediate) adjust for team, tools, reliability | Driver ratings are **subjective** |
| COCOMO II adds reuse, modern processes, early prototyping | Built on older projects; misses agile, cloud, open-source reuse |

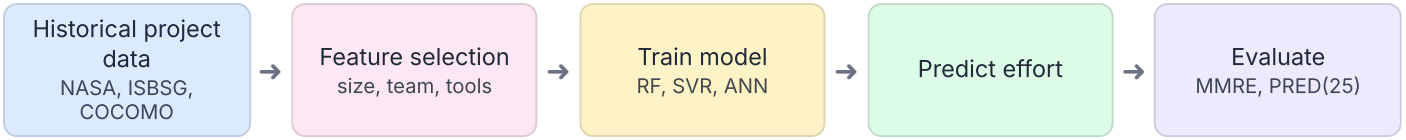
- **Conclusion:** good for **early, rough estimates (about ±20–40% when calibrated)**; not for precise planning.

## Q3 What factors have the greatest influence on software project cost estimation?

| Factor | Why it matters | Impact |
|---|---|---|
| **Software size** (KLOC / FP) | Effort grows faster than size (exponent > 1) | **Very high** |
| **Team capability & experience** | Analyst/programmer skill changes productivity many times over | **Very high** |
| **Requirement stability** | Changes cause rework | **High** |
| **Complexity & reliability needs** | Safety/critical systems need more design & testing | **High** |
| **Tools, technology & platform** | Good tools / reuse reduce effort | Medium |

| | | |
|---|---|---|
| **Schedule pressure** | Compressing time increases cost | Medium |
| **Process maturity & risk** | Well-defined process = predictable cost | Medium |
| **Team location & communication** | Distributed teams add overhead | Low–Medium |

## Q4 How can machine learning techniques improve software effort estimation accuracy?




- **Learns non-linear relationships** between size, team and cost that formulas like COCOMO cannot capture.
- **Techniques:** regression, **Random Forest / Gradient Boosting**, **Neural Networks**, **SVR**, analogy-based (**k-NN**), fuzzy logic, **ensembles**.
- **Auto-calibrates** to the organisation's own data and can be **retrained continuously**.
- **Ensembles + feature selection** reduce error and overfitting; **NLP** can estimate story points from requirement text.
- **Limits:** needs **good-quality data**, small datasets overfit, models are **black-box** (hard to explain).

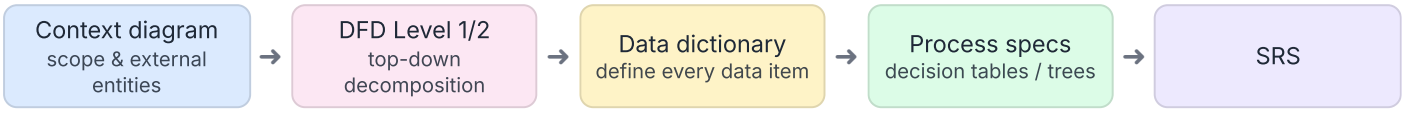
## Q5 Identify systems which are not information systems.

- **Information system (IS)** = people + procedures + data + technology that **collect, process, store and distribute information** to support decisions.
- **Not an IS** – systems whose purpose is not providing information:

| Category | Examples | Reason |
|---|---|---|
| **Natural systems** | Solar system, ecosystem, weather, human digestive system | No information processing for users |
| **Mechanical / physical** | Bicycle, car engine, mechanical watch, washing machine, building | Perform physical work only |
| **Pure control systems** | Simple thermostat, traffic-light timer, electric fan regulator | Control a physical process, not information for decisions |
| **Power / transport** | Electricity grid, water pipeline, railway track | Move energy / material |

- **Test:** if the system does not turn **data into information for people**, it is not an information system.

## Q6 How can structured analysis improve the identification of system requirements?




- **Graphical & top-down** (DFD, ER, state diagrams) → users and analysts understand the system together.
- **Context diagram** fixes **scope** and all external entities early.
- **Decomposition + balancing** exposes **missing inputs, outputs and data stores**.
- **Data dictionary** removes ambiguity; **process specs** give precise logic.
- Separates **logical (what)** from **physical (how)** design, giving a clean, **verifiable** requirement base.

## Q7 What techniques can be used to identify functional and non-functional requirements?

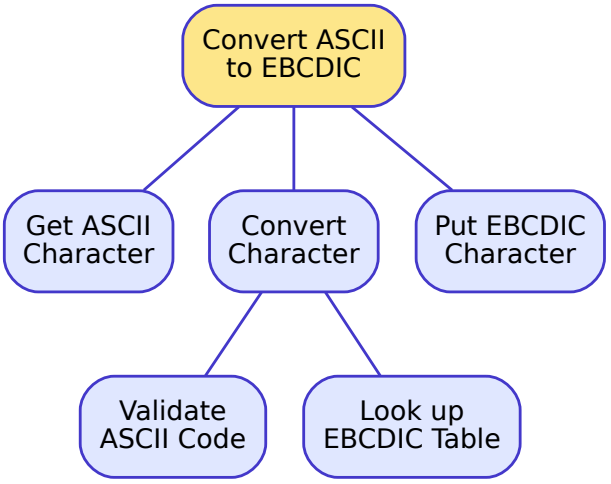
- **Functional (FR):** what the system **does** (e.g., "issue a book"). **Non-functional (NFR):** **how well** it does it (performance, security, usability, reliability).

| Technique | Use | FR | NFR |
|---|---|---|---|
| **Interviews** | Direct talk with stakeholders | ✓ | ✓ |
| **Questionnaires / surveys** | Many users, quick data | ✓ | ✓ |
| **Observation** | Watch real work (ethnography) | ✓ | ✓ (usability) |
| **Document analysis** | Forms, reports, existing system | ✓ | ✓ |

| | | | |
|---|---|---|---|
| **Brainstorming / JAD workshops** | Group agreement, conflict solving | ✓ | ✓ |
| **Prototyping** | Show screens, get feedback | ✓ | ✓ (usability) |
| **Use cases / user stories** | Scenario-based capture | ✓ | – |
| **Quality checklists / benchmarking** (ISO 25010) | Measurable quality targets | – | ✓ |

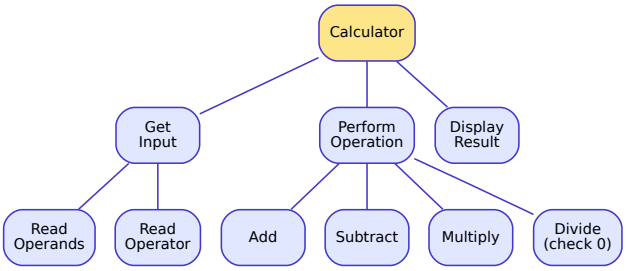
## Q8 Use structured design methodology to produce a design for the following systems.

### (a) ASCII to EBCDIC converter



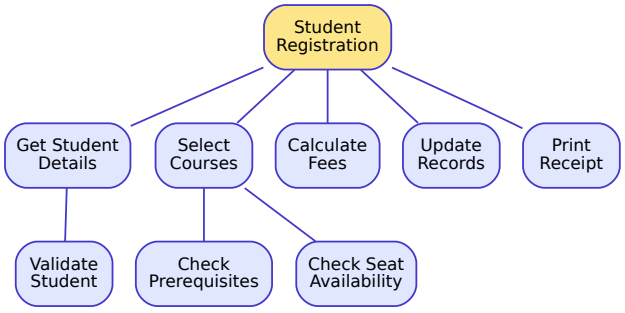

**Get** input → **Convert** (validate code, table look-up) → **Put** output.

### (b) Basic calculator



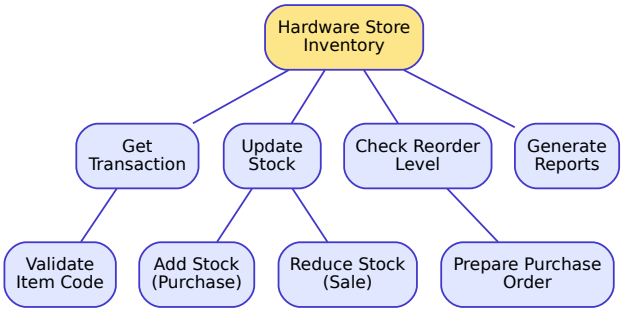

**Get** operands/operator → **Perform** + − × ÷ (check divide-by-0) → **Display**.

### (c) Student registration




**Get** & validate student → **Select** courses (prerequisite, seats) → fees → update → receipt.

### (d) Hardware-store inventory




**Get** sale/purchase → **Update stock** → **Reorder check** → reports.

- Each design is a **structure chart**: a **top-level control module** calls **get (afferent) → process (transform) → put (efferent)** modules.
- Goal: **high cohesion, low coupling**, one function per module.

## Q9 Design an information system for a college to manage student records.

### (a) Inputs, Processes, Outputs

**INPUTS**

student details, course choice, marks, attendance, fee payment

→

**PROCESSES**

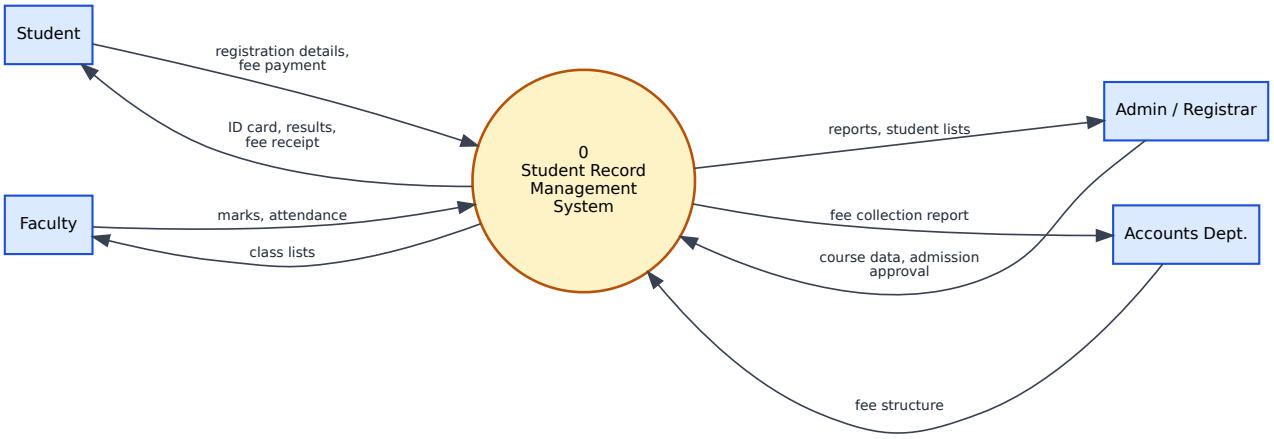
admission, enrollment, exam & result processing, fee management, reporting

→

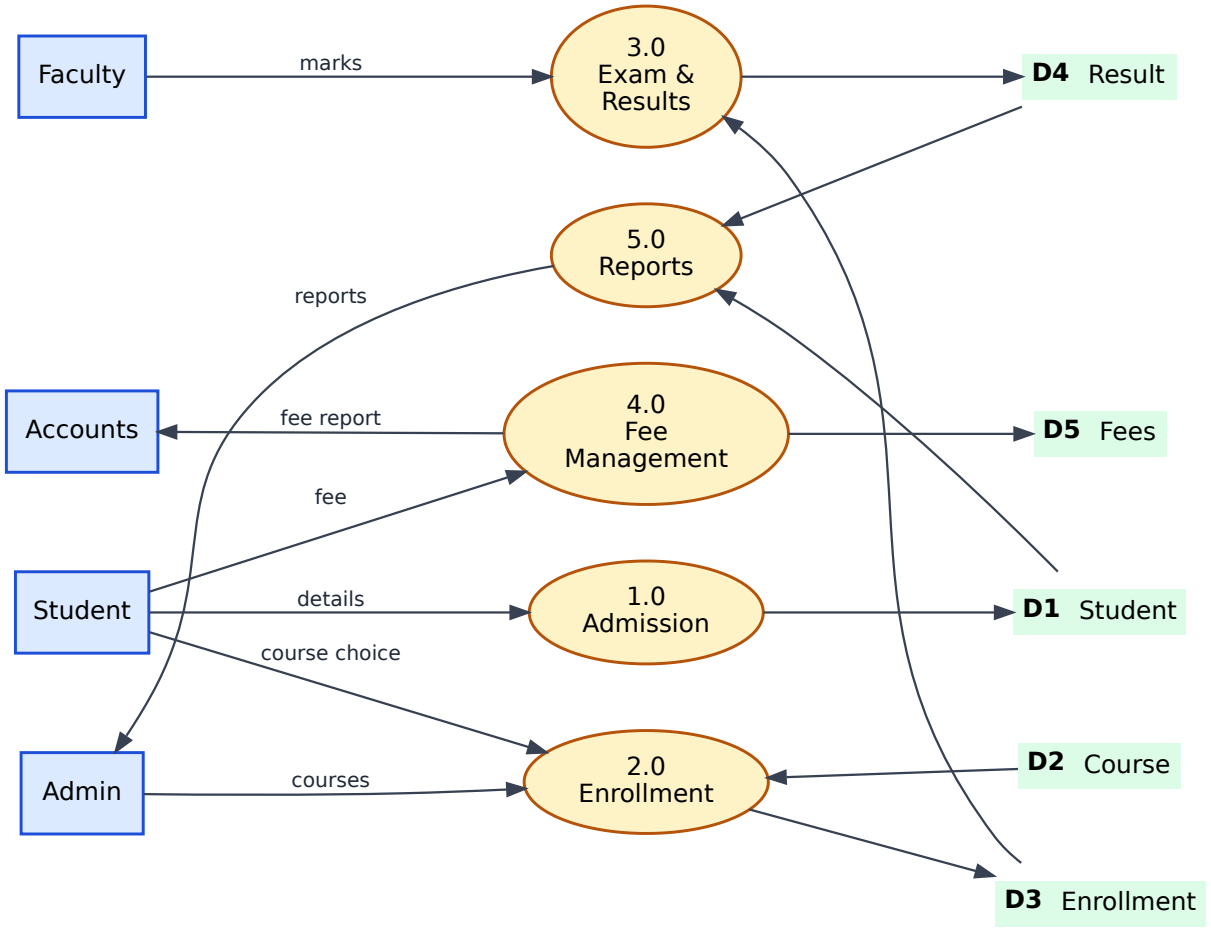
**OUTPUTS**

ID card, grade card, fee receipt, class lists, reports

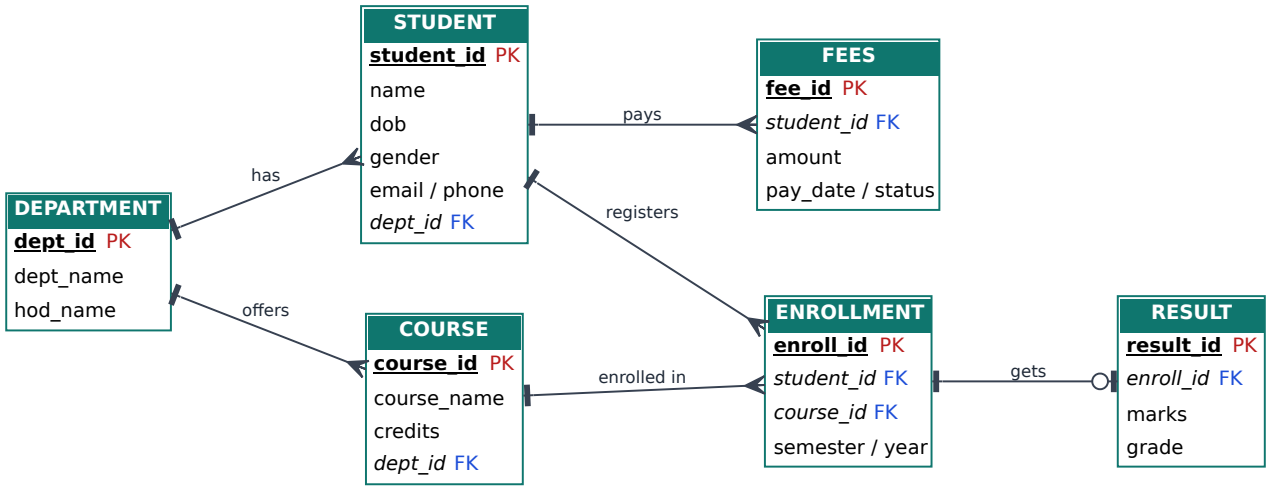
## (b) Context Diagram




## (c) Data Flow Diagram — Level 1



### (d) ER Diagram




*PK = primary key (underlined), FK = foreign key (italic); crow's foot = many*

### (e) Database Tables

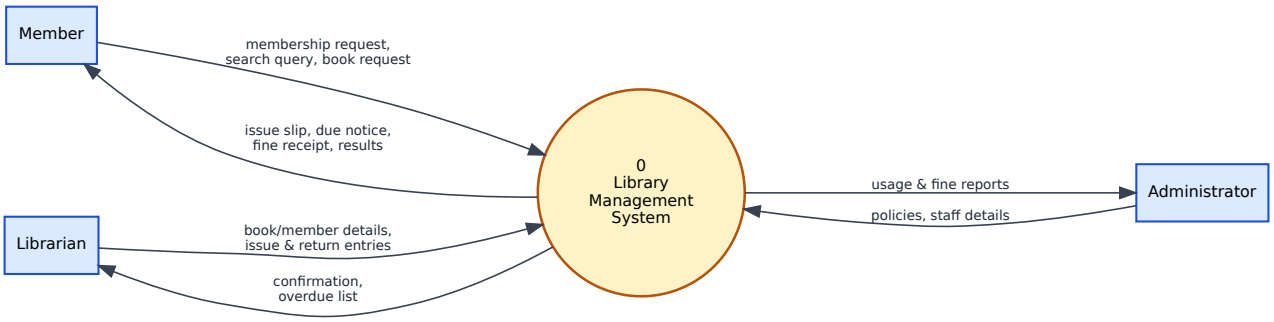
| Table | Columns (PK underlined, *FK* italic) |
|---|---|
| **DEPARTMENT** | dept_id, dept_name, hod_name |
| **STUDENT** | student_id, name, dob, gender, email, phone, *dept_id* → DEPARTMENT |
| **COURSE** | course_id, course_name, credits, *dept_id* → DEPARTMENT |
| **ENROLLMENT** | enroll_id, *student_id* → STUDENT, *course_id* → COURSE, semester, year |
| **RESULT** | result_id, *enroll_id* → ENROLLMENT, marks, grade |
| **FEES** | fee_id, *student_id* → STUDENT, amount, pay_date, status |

## Q10 Design an information system for a library.

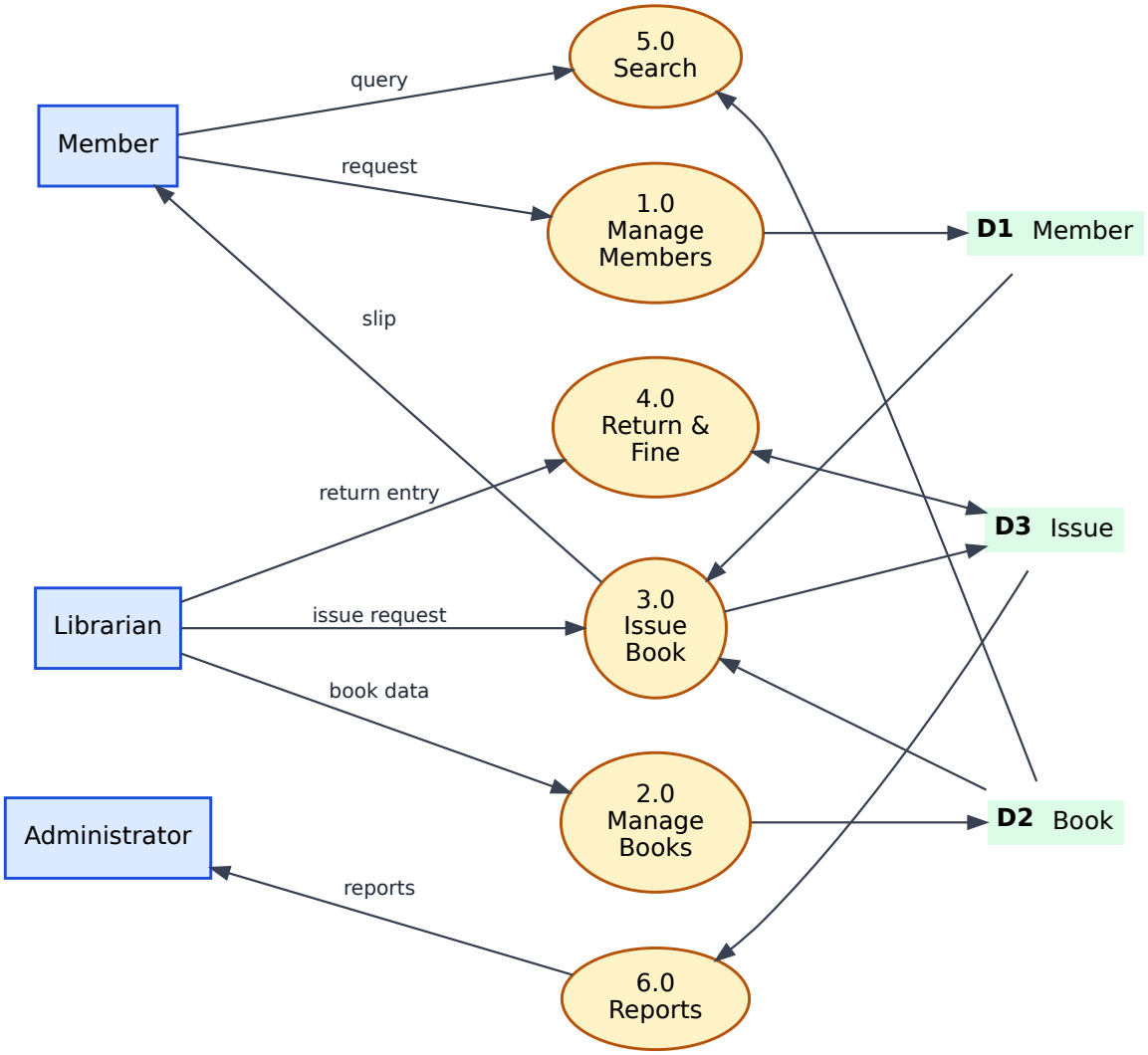
### (a) System Requirements

| Functional | Non-functional |
|---|---|
| Add / update / delete **books** | **Security:** role-based login (member / librarian / admin) |
| Register and manage **members** | **Performance:** catalogue search in < 3 s |
| **Search** catalogue (title, author, ISBN) | **Reliability:** daily backup, no data loss |
| **Issue, return, renew** books | **Usability:** simple UI, minimal training |
| Calculate **fine** for late return | **Scalability:** support growing books/members |
| Generate **reports** (issued, overdue, stock) | **Accuracy:** consistent stock counts |

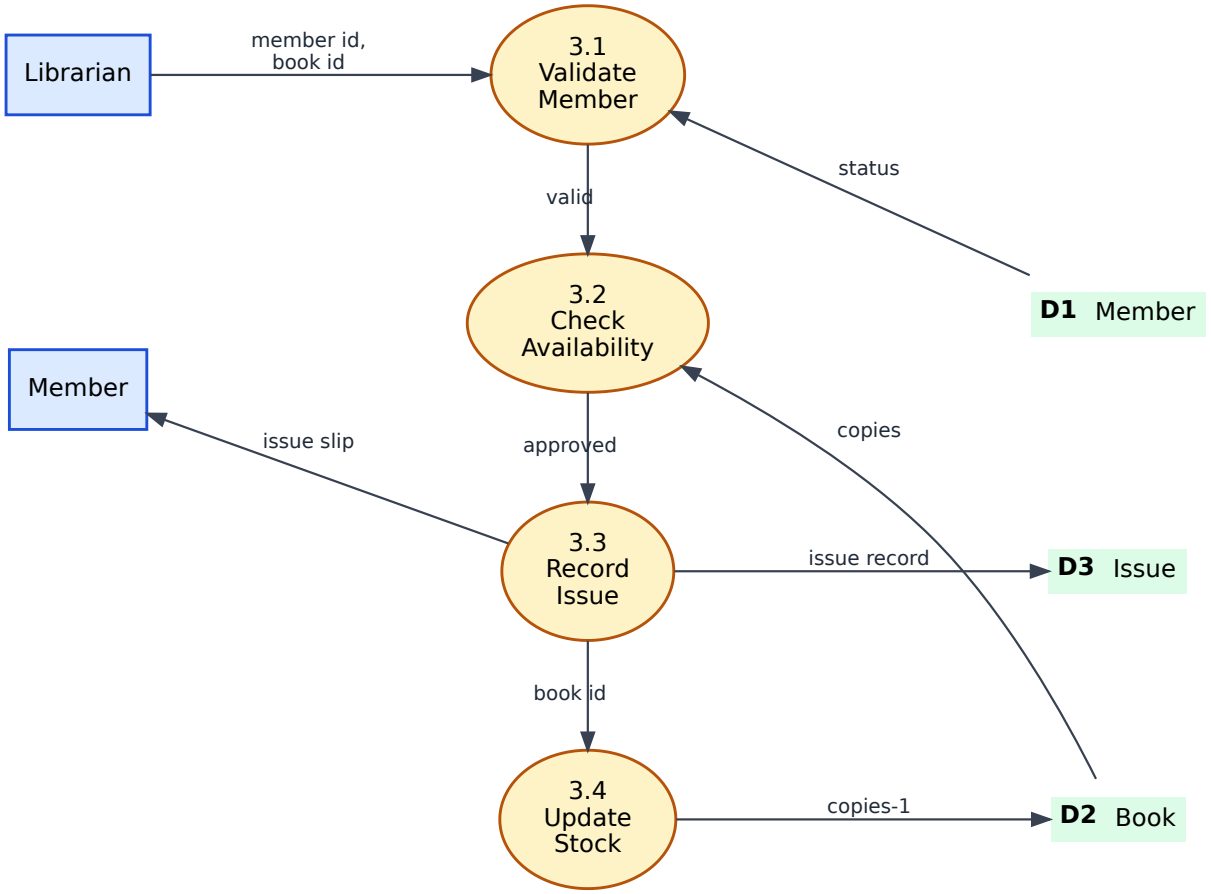
### (b) Context Diagram



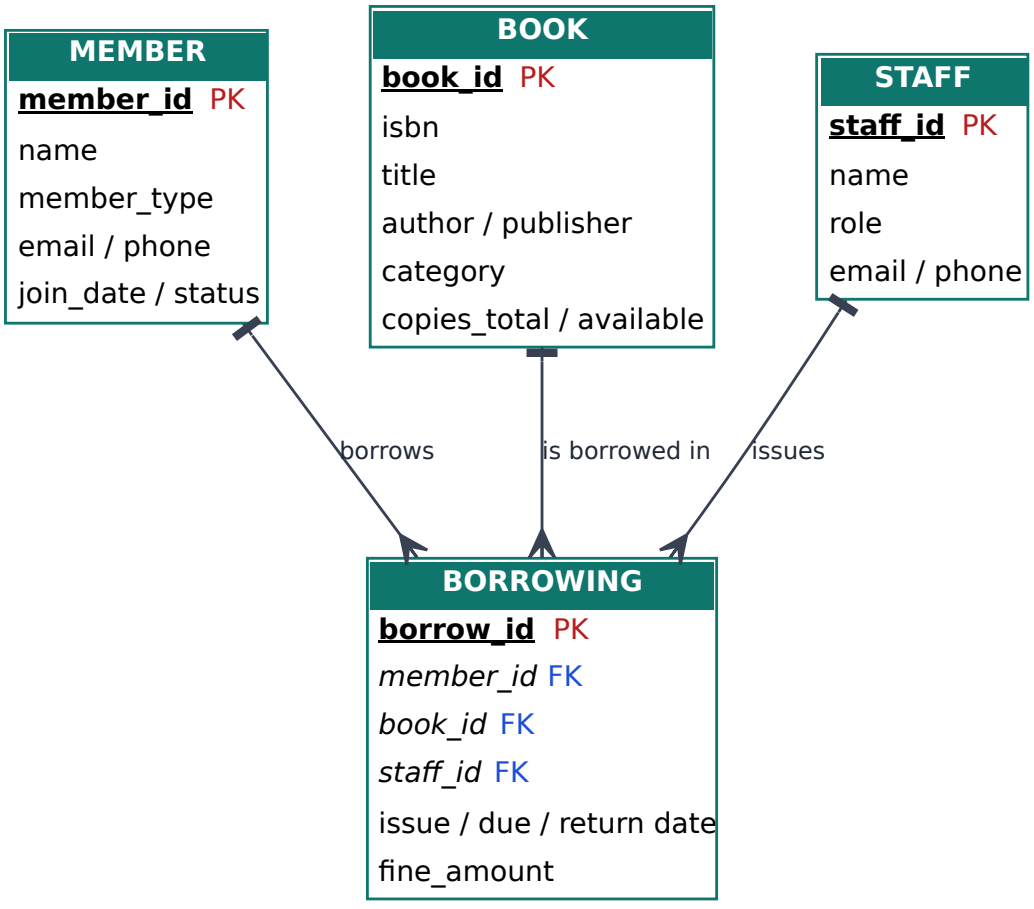
### (c) DFD Level 1


5.0
Search
query
Member
request
1.0
Manage
Members
D1 Member
slip
4.0
Return &
Fine
return entry
D3 Issue
Librarian
issue request
3.0
Issue
Book
book data
2.0
Manage
Books
D2 Book
Administrator
reports
6.0
Reports

### (d) DFD Level 2 — Process 3 (Issue Book)



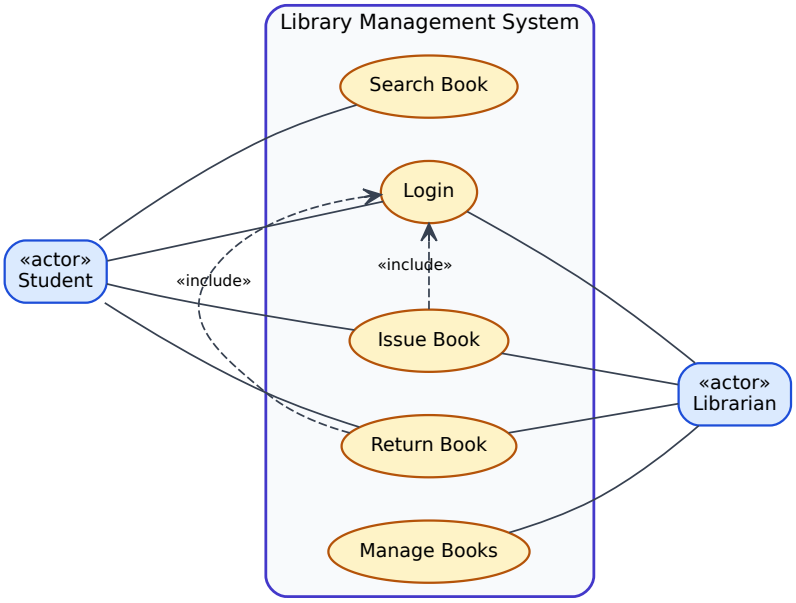

### (e) ER Diagram



### (f) Tables

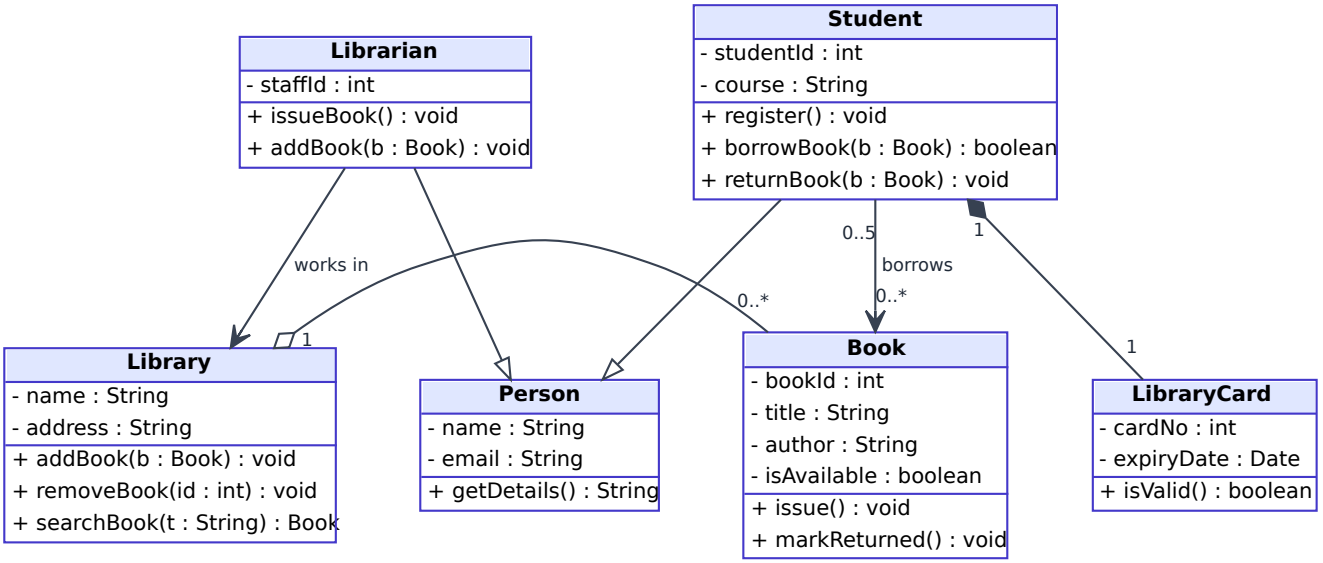
| Table | Columns (PK, *FK*) |
|---|---|
| **BOOK** | book_id, isbn, title, author, publisher, category, copies_total, copies_available |
| **MEMBER** | member_id, name, member_type, email, phone, join_date, status |
| **STAFF** | staff_id, name, role, email, phone |
| **BORROWING** | borrow_id, *member_id* → MEMBER, *book_id* → BOOK, *staff_id* → STAFF, issue_date, due_date, return_date, fine_amount |

## Q11 Create a Use Case Diagram (Library Management System).



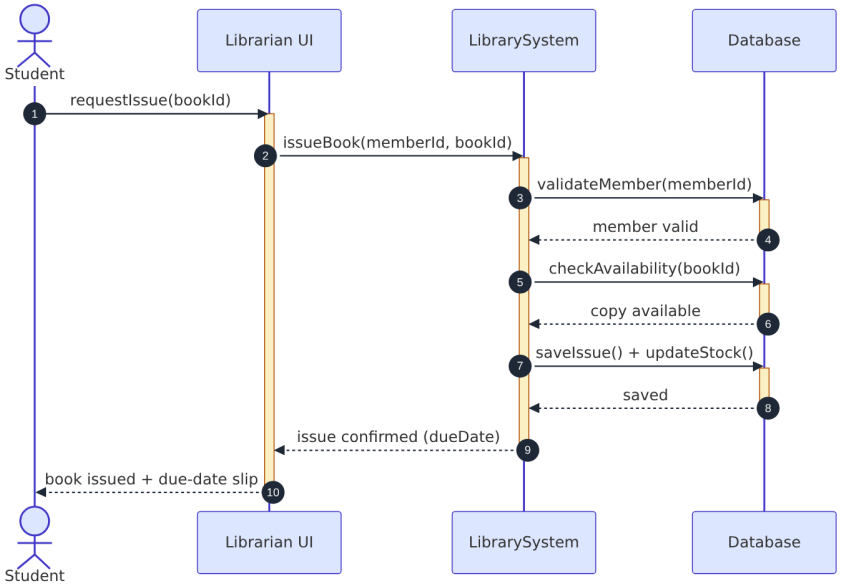

- **System:** Library Management System (boundary box).
- **Actors: Student**, **Librarian**.
- **Use cases: Login**, **Issue Book**, **Return Book** (+ Search Book, Manage Books).
- **Associations:** solid lines actor ↔ use case.
- **«include»:** Issue Book and Return Book always include Login.

## Q12 Draw a Class Diagram (Student, Book, Library).




| Relationship | Notation | In this diagram |
|---|---|---|
| **Inheritance** | Hollow triangle | Student, Librarian **are-a** Person |
| **Aggregation** | Hollow diamond | Library **has** Books (books can exist independently) |
| **Composition** | Filled diamond | Student **owns** LibraryCard (card dies with student) |
| **Association** | Plain arrow | Student **borrows** Book; Librarian **works in** Library |

## Q13 Create a Sequence Diagram for the "Book Issue" process.




- **Objects:** Student, Librarian UI, LibrarySystem, Database.
- **Messages:** solid arrows (calls); **return messages:** dashed arrows; **activation bars:** thin rectangles on lifelines.
- **Flow:** validate member → check availability → save issue & update stock → confirm with due date.

## Q14 How accurately can AI generate test cases from software requirements?

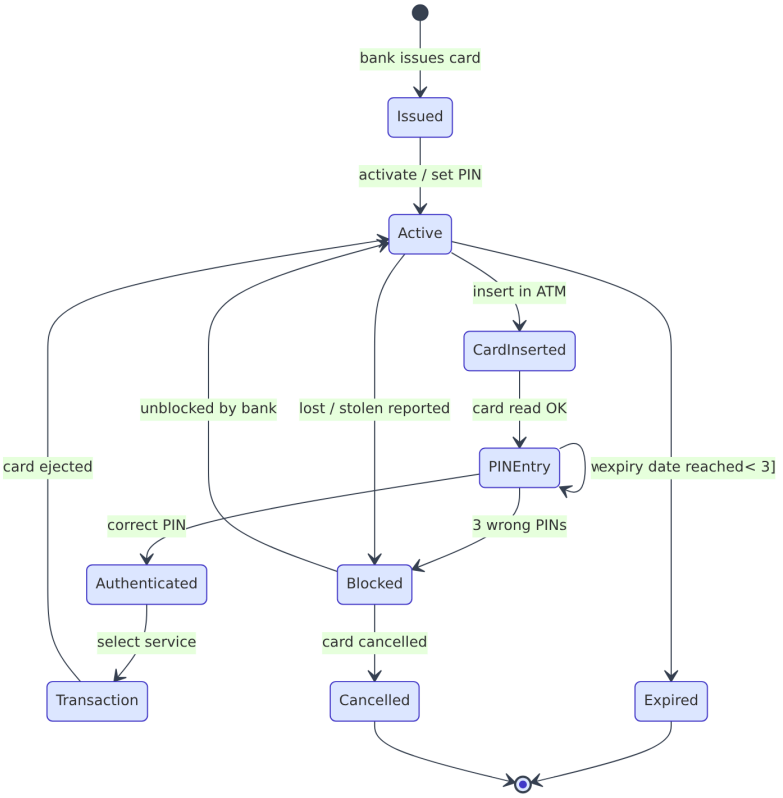
| Where AI does well | Where AI is weak |
|---|---|
| Quickly generates **positive, negative, boundary** cases | **Wrong expected results** (hallucinated oracle) |
| Good **coverage of obvious scenarios** from clear requirements | Misses **implicit domain rules** and complex business logic |
| Saves time; consistent format; helps beginners | Produces **duplicates** / untestable steps |
| Can read use cases, user stories, SRS text | Accuracy drops with **vague or incomplete** requirements |

- **Accuracy is moderate and varies** with requirement quality, prompt and model – AI output is a **first draft**, not final.
- **Improve:** clear requirements, **prompt templates / few-shot examples**, add domain context (RAG), **human review**.
- **Measure:** test **validity**, requirement coverage, **mutation score**, defects found.

## Q15 Major challenges in implementing automated testing in small software organizations.

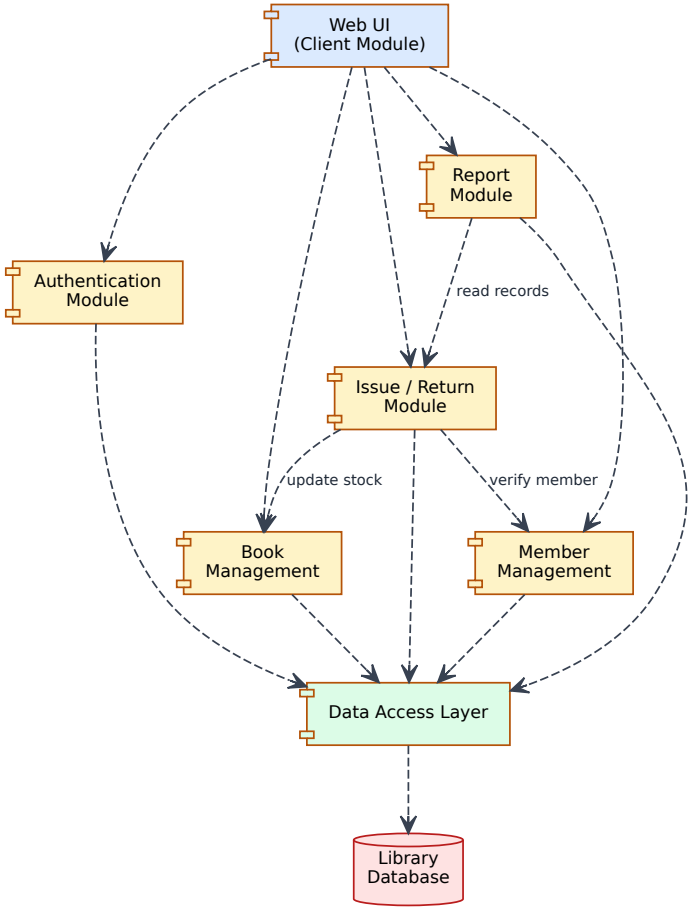
| Challenge | Possible solution |
|---|---|
| **Limited budget** (tools, licences, infra) | Use **open-source** (Selenium, Cypress, JUnit, pytest) |
| **Skill shortage** in automation | Training, pair work, simple frameworks |
| **Tight deadlines** / small team multitasks | Automate only **critical & regression** tests first |
| **Frequent requirement changes** → script maintenance | Modular scripts, **page-object** design |
| **Unclear ROI** & weak management support | Pilot project, show time saved |
| **No test strategy / data / environment** | Define plan, use **CI**, containers for environments |
| **Flaky tests** and legacy code | Stabilise waits, refactor for testability |

## Q16 Draw a State Chart Diagram for the states of an ATM card.



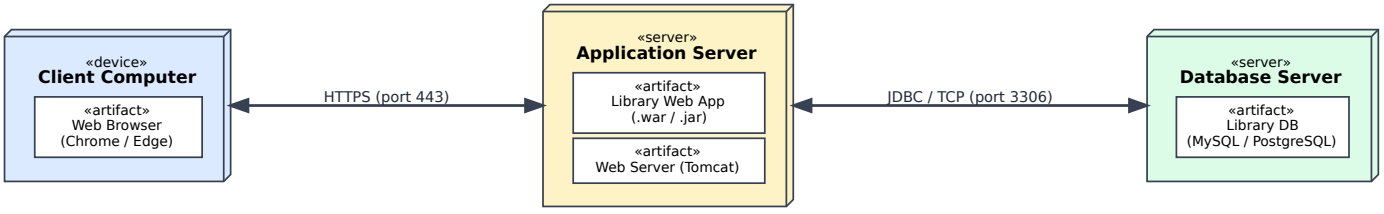

- **Main flow:** Issued → Active → CardInserted → PINEntry → Authenticated → Transaction → (card ejected) Active.
- **Self-loop:** wrong PIN while attempts < 3. **3 wrong PINs** or **lost/stolen** → **Blocked**.
- **End states:** Expired, Cancelled.

## Q17 Create a Component Diagram (software modules and dependencies).




- **Web UI** depends on Authentication, Member, Book, Issue/Return and Report modules.
- **Issue/Return** depends on Member (verify) and Book (update stock); **Report** reads Issue records.
- All modules use the **Data Access Layer** → **Library Database** (layered, loosely coupled).

## Q18 Create a Deployment Diagram (Client, Application Server, Database Server).




- **Nodes:** Client computer (browser), **Application server** (web app + Tomcat), **Database server** (MySQL/PostgreSQL).
- **Communication links:** Client ↔ App server over **HTTPS**; App server ↔ DB server over **JDBC/TCP**.

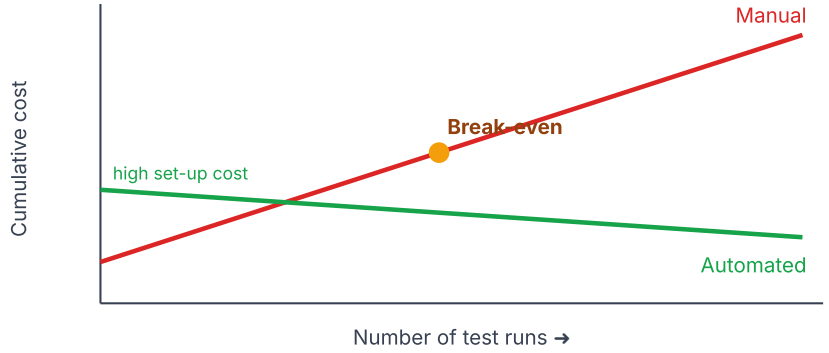
## Q19 How effective are automated tools in validating Software Requirements Specifications (SRS)?

| Effective at (syntax / quality) | Not effective at (meaning / intent) |
|---|---|
| **Ambiguity** words: “fast”, “user-friendly”, “etc.” | Whether requirement is **correct** for stakeholders |
| **Incompleteness** (TBD, missing sections) | **Missing** requirements nobody wrote |
| **Inconsistency**, duplicates, conflicting terms | **Feasibility** and business value |
| **Traceability** gaps, IEEE 830 / ISO 29148 checks | Domain-specific logic errors |

- **Tools:** DOORS, Jama, ReqView, NLP analyzers (e.g., QuARS), model checkers.
- **Verdict: highly effective for quick, consistent quality checks**, but must be combined with **manual reviews / inspections**.

## Q20 Compare automated and manual testing in terms of defect detection and testing time.

| Aspect | Automated testing | Manual testing |
|---|---|---|
| **Defect detection** | Best for **regression** defects; consistent, no human slip | Best for **usability, look & feel, new/unexpected** defects (exploratory) |
| **Testing time** | **Fast execution**, runs 24×7 — but **script creation is slow** | No set-up, but **slow and repetitive** for many runs |
| **Cost** | High initial, low per run | Low initial, grows with every run |
| **Accuracy** | Repeatable, same steps every time | Prone to human error / fatigue |
| **Best use** | Regression, load, smoke, data-driven tests | Ad-hoc, UI/UX, short-lived projects |




*Automation costs more at start but becomes cheaper after repeated runs*

- **Best practice: automate repetitive tests, keep exploratory testing manual**.

## Q21 Compare equivalence partitioning with boundary value analysis in detecting input-related errors.




*Rule: age must be 18–60 — ● EP picks one value per class, ● BVA tests values at the edges*

| Aspect | Equivalence Partitioning (EP) | Boundary Value Analysis (BVA) |
|---|---|---|
| **Idea** | Divide inputs into **classes**; test one value per class | Test values **at and around edges** of classes |
| **Detects** | **Class-level** errors (wrong handling of a whole range) | **Off-by-one**, < vs ≤ errors at boundaries |
| **No. of tests** | Few (reduces redundancy) | More (typically 2–3 per boundary) |
| **Weakness** | Misses boundary bugs | Misses errors inside the range |
| **Example** (age 18–60) | 10, 35, 75 | 17, 18, 19, 59, 60, 61 |

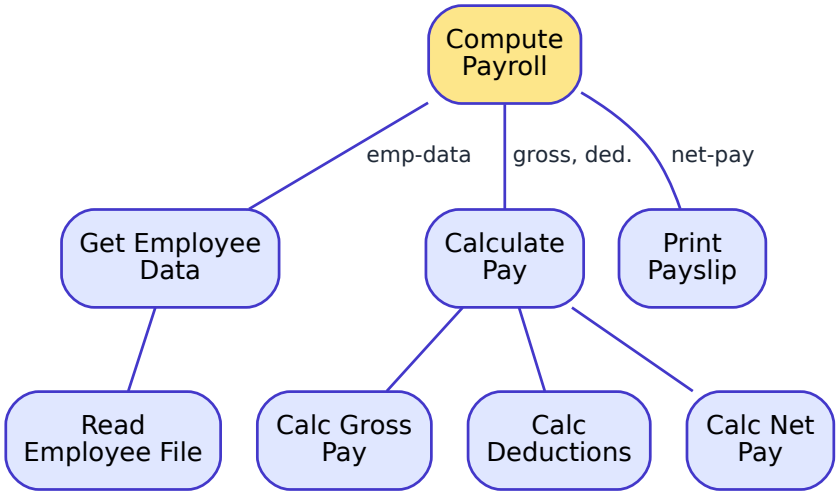
- **Conclusion:** BVA catches more input errors per test, EP reduces test count — **use both together**.

## Q22 Explain structured design methodology with the help of an example.

- **Structured design** (Yourdon & Constantine) converts the **DFD from analysis** into a **hierarchical structure chart** of modules.
- **Principles: modularity, high cohesion, low coupling**, top-down decomposition, controlled fan-in / fan-out.

1. Study DFD → 2. Find **central transform** (afferent / efferent flows) → 3. Draw first-cut **structure chart** → 4. Refine (cohesion, coupling) → 5. Verify vs requirements

**Example: Payroll system**




*Structure chart — top module **Compute Payroll** calls Get Employee Data (afferent), Calculate Pay (transform), Print Payslip (efferent); labels show data passed between modules*

- **Afferent** = brings data in; **transform** = computes; **efferent** = sends results out.
- **Benefits:** easy to code, test and maintain; modules are reusable.